Exercise 1: Resource Files and Variable Encapsulation Goal: Create a clean separation between locators and test logic. The Task: Create a `login_resources.resource` file. Implementation: Define `${USERNAME_FIELD}`, `${PASSWORD_FIELD}`, and `${LOGIN_BUTTON}` in the ` Variables ` section. Create a keyword `Login To Application` that takes `${username}` and `${password}` as arguments. The Test: Write a test case in a `.robot` file that imports the resource and logs in. Verification: Verify that the "Products" title is visible.

**login_resources.resource**

```
*** Settings ***
Library    SeleniumLibrary

*** Variables ***
${URL}              https://www.saucedemo.com
${BROWSER}          chrome

${USERNAME_FIELD}    id:user-name
${PASSWORD_FIELD}    id:password
${LOGIN_BUTTON}      id:login-button
${PRODUCTS_TITLE}    xpath://span[text()='Products']

*** Keywords ***
Open Login Page
   Open Browser    ${URL}    ${BROWSER}
   Maximize Browser Window

Login To Application
   [Arguments]    ${username}    ${password}

   Input Text       ${USERNAME_FIELD}    ${username}
   Input Password    ${PASSWORD_FIELD}    ${password}
   Click Button      ${LOGIN_BUTTON}

Verify Products Page
```

```
    Element Should Be Visible    ${PRODUCTS_TITLE}

Close Browser Session
    Close Browser
```

**login_test.robot**

```
*** Settings ***
Resource    login_resources.resource

Suite Setup       Open Login Page
Suite Teardown    Close Browser Session

*** Test Cases ***
Valid Login Test
    Login To Application    standard_user    secret_sauce

    Verify Products Page
```

Exercise 2: Custom Python Libraries for POM Goal: Extend Robot Framework with a Python-based Page Object. The Task: Write a Python class `ProductPage.py`. Inside, create a method `get_product_price_by_name(product_name)` that returns the price of a specific item. Implementation: Import this Python file as a `Library` in your Robot script. The Test: Log in, then use your custom keyword to fetch the price of "Sauce Labs Backpack" and "Sauce Labs Bike Light". Verification: Use `Should Be Equal As Numbers` to verify the total sum of these two items is correct.

**ProductPage.py**

```
from selenium.webdriver.common.by import By
from robot.libraries.BuiltIn import BuiltIn
```

```
class ProductPage:

    def get_product_price_by_name(self, product_name):

        seleniumlib = BuiltIn().get_library_instance('SeleniumLibrary')
        driver = seleniumlib.driver

        products = driver.find_elements(By.CLASS_NAME, "inventory_item")

        for product in products:

            name = product.find_element(By.CLASS_NAME,
"inventory_item_name").text

            if name == product_name:

                price_text = product.find_element(
                    By.CLASS_NAME,
                    "inventory_item_price"
                ).text

                price = price_text.replace("$", "")

                return float(price)

        raise Exception(f"Product not found: {product_name}")
```

**product_test.robot**

```
*** Settings ***
Library     SeleniumLibrary
Library     ProductPage.py

*** Variables ***
```

```
${URL}              https://www.saucedemo.com
${BROWSER}          chrome

${USERNAME_FIELD}     id:user-name
${PASSWORD_FIELD}     id:password
${LOGIN_BUTTON}       id:login-button

*** Test Cases ***
Verify Product Total Price

    Open Browser    ${URL}    ${BROWSER}
    Maximize Browser Window

    Input Text       ${USERNAME_FIELD}    standard_user
    Input Password    ${PASSWORD_FIELD}    secret_sauce
    Click Button      ${LOGIN_BUTTON}

    ${backpack_price}=      Get Product Price By Name    Sauce Labs Backpack
    ${bike_light_price}=    Get Product Price By Name    Sauce Labs Bike Light

    ${total}=    Evaluate    ${backpack_price} + ${bike_light_price}

    Should Be Equal As Numbers    ${total}    39.98

    Close Browser
```

Exercise 3: Setup/Teardown with Screenshots Goal: Manage browser lifecycle and failure evidence. The Task: Implement a `Suite Setup` to open the browser and a `Test Teardown` to capture evidence. Implementation: In the teardown, use the `Run Keyword If Test Failed` keyword to trigger `Capture Page Screenshot`. Challenge: Configure the screenshot name to include the test name and a timestamp (e.g., `failure_${TEST_NAME}.png`).

**setup_teardown_test.robot**

```
*** Settings ***
Library    SeleniumLibrary
Library    DateTime

Suite Setup       Open Browser Session
Suite Teardown    Close Browser
Test Teardown     Capture Failure Evidence

*** Variables ***
${URL}              https://www.saucedemo.com
${BROWSER}          chrome

${USERNAME_FIELD}     id:user-name
${PASSWORD_FIELD}     id:password
${LOGIN_BUTTON}       id:login-button

*** Keywords ***
Open Browser Session
   Open Browser    ${URL}    ${BROWSER}
   Maximize Browser Window

Capture Failure Evidence
   ${timestamp}=    Get Current Date    result_format=%Y%m%d_%H%M%S

   Run Keyword If Test Failed
   ...    Capture Page Screenshot
   ...    failure_${TEST_NAME}_${timestamp}.png

*** Test Cases ***
Invalid Login Test

   Input Text       ${USERNAME_FIELD}    standard_user
   Input Password    ${PASSWORD_FIELD}    wrong_password
   Click Button      ${LOGIN_BUTTON}

   Page Should Contain    Products
```

Exercise 4: Data-Driven Testing with Scenario Outlines Goal: Test multiple login credentials using a Template. The Task: Use the `Test Template` feature in Robot Framework. Implementation: Create a keyword `Invalid Login Scenario`. The Data: Provide a table of `invalid_user`, `locked_out_user`, and `problem_user` with their respective error messages. Verification: Ensure that for each row, the correct error message (e.g., "Epic sadface: Username and password do not match any user in this service") appears.

**data_driven_test.robot**

```
*** Settings ***
Library    SeleniumLibrary

Test Template    Invalid Login Scenario

Suite Setup       Open Browser Session
Suite Teardown    Close Browser

*** Variables ***
${URL}              https://www.saucedemo.com
${BROWSER}          chrome

${USERNAME_FIELD}     id:user-name
${PASSWORD_FIELD}     id:password
${LOGIN_BUTTON}       id:login-button
${ERROR_MESSAGE}      xpath://h3[@data-test='error']

*** Keywords ***
Open Browser Session
   Open Browser    ${URL}    ${BROWSER}
   Maximize Browser Window

Invalid Login Scenario
   [Arguments]    ${username}    ${password}    ${expected_error}

   Go To    ${URL}
```

```
    Input Text       ${USERNAME_FIELD}    ${username}
    Input Password    ${PASSWORD_FIELD}    ${password}
    Click Button      ${LOGIN_BUTTON}

    Element Text Should Be    ${ERROR_MESSAGE}    ${expected_error}

*** Test Cases ***
Invalid User Login
    invalid_user
    secret_sauce
    Epic sadface: Username and password do not match any user in this service

Locked Out User Login
    locked_out_user
    secret_sauce
    Epic sadface: Sorry, this user has been locked out.

Problem User Login
    problem_user
    wrong_password
    Epic sadface: Username and password do not match any user in this service
```

Exercise 5: Advanced Reporting with Robot & Allure Goal: Integrate Allure for high-level visualization. The Task: Install the `robotframework-allure` listener. Implementation: Add tags to your test cases like `[Tags] Critical Smoke Regression`. Execution: Run your tests using the command line flag to generate Allure results: `robot --listener allure_robotframework .` Challenge: Use the `Set Test Documentation` keyword within your test to provide a detailed description that will show up in the Allure report dashboard.

**Command**

pip install robotframework-allure

**allure_test.robot**

```
*** Settings ***
Library    SeleniumLibrary

*** Variables ***
${URL}              https://www.saucedemo.com
${BROWSER}          chrome

${USERNAME_FIELD}     id:user-name
${PASSWORD_FIELD}     id:password
${LOGIN_BUTTON}       id:login-button
${PRODUCTS_TITLE}     xpath://span[text()='Products']

*** Test Cases ***
Valid Login Test

   [Tags]    Critical    Smoke    Regression

   Set Test Documentation
   ...    Verify successful login functionality using valid credentials and validate
Products page visibility.

   Open Browser    ${URL}    ${BROWSER}
   Maximize Browser Window

   Input Text       ${USERNAME_FIELD}    standard_user
   Input Password    ${PASSWORD_FIELD}    secret_sauce
   Click Button      ${LOGIN_BUTTON}

   Element Should Be Visible    ${PRODUCTS_TITLE}

   Close Browser
```

**Run Tests with Allure Listener**

**Command**

robot --listener allure_robotframework .

**Generate Allure Report**

**Command**

allure serve output/allure